

**Arquitectura y Modelo de Datos de una Plataforma de Big Data Simulada
para la Inteligencia de Mercado de Videojuegos Free-to-Play**

Proyecto Integrador de Big Data: ingesta, preprocesamiento ELT y enriquecimiento de datos

Documentación de la Arquitectura y Modelo de Datos

Felipe Fernández Rodríguez

Ingeniería en Software y Datos, Institución Universitaria Digital de Antioquia

PREICA2602B010136 - Infraestructura y arquitectura para Big Data

Cesar Luis Vasquez Suarez

Rionegro, Antioquia

27 de septiembre del 2026

Tabla de Contenido

Resumen.....	5
Introducción	6
Contexto del Proyecto.....	7
Escenario Organizacional (Caso de Estudio Simulado)	7
Problema y Objetivo Analítico	8
Alcance	8
Evolución del Pipeline	9
Arquitectura de la Plataforma	9
Visión Global	9
Componentes Principales.....	11
Simulación del Entorno Cloud.....	12
Flujo de Datos	13
Capa de Ingesta	13
Capa de Preprocesamiento (ELT).....	15
Extracción y Carga.....	16
Perfilamiento.....	16
Transformación	17
Validación Final y Puntaje de Calidad.....	18
Capa de Enriquecimiento	20

Homologación por Título..... 20

Resolución de Duplicados y Unión..... 21

Control de Cardinalidad y Validación Final 22

Flujo End-to-End 23

Automatización y Control de Versiones 24

 GitHub como Repositorio Único 24

 Flujos de Trabajo de GitHub Actions 24

 Pruebas Automatizadas 25

 Evidencias y Artifacts 26

Modelo de Datos 26

 Modelo Físico: Base de Datos SQLite..... 27

 Modelo Físico, Conjunto Procesado y Conjunto Enriquecido..... 28

 Diagrama del Modelo de Datos 28

 Modelo Lógico del Conjunto Enriquecido 29

 Claves y Relaciones 31

 Linaje de Datos 32

 Justificación del Modelo de Datos 33

Herramientas y Tecnologías 34

 Justificación de PySpark en un Entorno Local 35

Calidad y Validación..... 36

Seguridad y Configuración	37
Evidencias Generadas	37
Limitaciones.....	39
Recomendaciones y Evolución hacia la Nube	40
Conclusiones	42
Referencias.....	44

Resumen

Este documento describe la arquitectura, el flujo de datos, la automatización y el modelo de datos de una plataforma de Big Data simulada en un entorno local, construida para consolidar información sobre el mercado de videojuegos *free-to-play*. La plataforma se estructura en tres capas de procesamiento encadenadas: una capa de ingesta que extrae el catálogo público de la API de FreeToGame y lo persiste en una base de datos analítica SQLite; una capa de preprocesamiento bajo el paradigma ELT (*Extract, Load, Transform*) que perfila, limpia, tipifica y valida los datos con PySpark en modo local; y una capa de enriquecimiento que integra, mediante homologación de títulos y uniones externas izquierdas (LEFT JOIN), dos fuentes adicionales en formato JSON: la API de giveaways de GamerPower y el catálogo de MMOBomb. Cada capa genera evidencias auditables (muestras en CSV y reportes en texto plano), cuenta con pruebas automatizadas y se ejecuta de forma reproducible mediante GitHub Actions. Se documenta el modelo físico (tabla `games` en SQLite), su relación con los conjuntos de datos derivados (`games_cleaned.csv` y `games_enriched.csv`), el linaje de cada columna, la justificación de las herramientas empleadas, las limitaciones del entorno simulado y una ruta de evolución hacia una infraestructura en la nube. El escenario organizacional presentado es un caso de estudio simulado; la implementación técnica, los datos consumidos y los artefactos descritos corresponden al repositorio real del proyecto.

Palabras clave: Big Data, ELT, PySpark, SQLite, enriquecimiento de datos, linaje de datos, GitHub Actions, modelo de datos.

Introducción

El mercado de los videojuegos *free-to-play* se caracteriza por una oferta amplia, dispersa entre múltiples catálogos y portales, y por una actividad promocional constante (regalos, paquetes de contenido, accesos anticipados) que cambia día a día. Para una organización que necesite entender ese mercado, la información relevante no se encuentra en una sola fuente: el catálogo de títulos vive en un proveedor, las promociones vigentes en otro y las descripciones alternativas del mismo juego en un tercero. Consolidar esas fuentes exige un proceso de ingeniería de datos que sea trazable, repetible y verificable, más que un ejercicio puntual de descarga y cruce de archivos.

Este documento presenta la arquitectura de una plataforma de datos construida con ese propósito. La plataforma simula, en un entorno local y con herramientas de código abierto, los componentes típicos de una arquitectura de Big Data en la nube: una zona de datos crudos (archivos JSON tal como los entregan las APIs), una capa de almacenamiento analítico (SQLite), un motor de procesamiento distribuido (PySpark ejecutado en modo `local[*]`), zonas de datos procesados y enriquecidos (archivos CSV versionados) y un mecanismo de orquestación y automatización (GitHub Actions). La decisión de simular localmente responde al alcance académico del proyecto: permite practicar y documentar el paradigma de procesamiento distribuido sin contratar infraestructura, manteniendo un diseño que podría migrarse a servicios administrados con cambios acotados (véase la sección de recomendaciones).

La plataforma se desarrolló de forma incremental en tres etapas que, en conjunto, conforman un único pipeline de datos: la ingesta desde la API de FreeToGame hacia SQLite; el preprocesamiento y la limpieza con un enfoque ELT sobre PySpark; y el enriquecimiento del conjunto limpio con las APIs de GamerPower y MMOBomb. Cada etapa reutiliza el código y las salidas de la anterior, comparte una configuración centralizada y deja evidencias de auditoría con

cifras calculadas durante la ejecución. El presente documento cierra el proyecto consolidando la descripción técnica de todo el recorrido de los datos, desde la fuente externa hasta el conjunto analítico final, y definiendo el modelo de datos resultante.

El objetivo del documento es, por tanto, triple: (a) explicar la arquitectura global y sus componentes, con diagramas de flujo por capa; (b) definir el modelo de datos físico y lógico, sus claves, relaciones y linaje; y (c) justificar las herramientas seleccionadas, identificando de forma transparente las limitaciones del entorno simulado y las oportunidades de evolución hacia una plataforma en la nube. Toda afirmación técnica se fundamenta en la implementación real contenida en el repositorio (código fuente, flujos de trabajo de GitHub Actions, pruebas automatizadas y archivos de configuración); cuando un requisito conceptual no coincide exactamente con lo implementado, se indica la diferencia y se propone como mejora futura en lugar de presentarlo como existente.

Contexto del Proyecto

Escenario Organizacional (Caso de Estudio Simulado)

Para dar sentido de negocio a la plataforma se adopta el escenario de *LudoMetrics Analytics*, una firma ficticia de inteligencia de mercado especializada en el segmento *free-to-play*. La firma asesora a estudios independientes y a equipos de marketing que necesitan responder preguntas como: ¿qué títulos componen la oferta actual por género y plataforma?, ¿cuáles de ellos tienen promociones o giveaways activos y con qué valor estimado?, ¿coincide la información de un título entre distintos portales o existen discrepancias de género, plataforma, distribuidor o desarrollador que deban revisarse antes de publicar un informe? Este escenario es una construcción académica: la organización no existe, pero las fuentes de datos, las APIs consultadas, el código y los artefactos que se documentan a continuación son reales y se ejecutan tal como se describen.

El requerimiento de LudoMetrics Analytics hacia el equipo de ingeniería de datos se resume en construir un pipeline que produzca, de manera automática y reproducible, un conjunto de datos analítico con granularidad de un registro por videojuego, que integre el catálogo base con la actividad promocional y con un catálogo de contraste, conservando la procedencia de cada atributo y dejando constancia auditable de cada transformación aplicada. Sobre ese conjunto se apoyarían los análisis y visualizaciones de mercado de la firma.

Problema y Objetivo Analítico

El problema técnico consiste en que las tres fuentes disponibles son sistemas independientes, con identificadores internos propios que no se corresponden entre sí, con estructuras de respuesta que pueden cambiar y con calidad de datos no garantizada (espacios sobrantes, campos vacíos, fechas en formatos potencialmente inválidos, títulos con anotaciones promocionales). El objetivo analítico de la plataforma es entregar un conjunto de datos confiable en el que cada juego del catálogo base aparezca exactamente una vez, con sus atributos originales intactos, ampliado con las columnas de las fuentes adicionales cuando exista una coincidencia verificable de título y con valores nulos explícitos cuando no la haya.

Alcance

La plataforma cubre el ciclo completo desde la extracción hasta la publicación del conjunto enriquecido y sus evidencias, incluyendo pruebas automatizadas y ejecución programada. Quedan fuera del alcance el análisis estadístico y la visualización sobre el conjunto final, así como el despliegue en infraestructura de nube real; ambos se tratan únicamente como recomendaciones de evolución. Los datos se procesan en formato JSON (respuestas de las APIs), SQLite (almacenamiento analítico) y CSV (conjuntos procesados y muestras); no se emplean otros formatos de intercambio.

Evolución del Pipeline

El pipeline se construyó en tres fases que se apoyan una en la otra sin reemplazar lo ya construido, tal como se muestra en la Tabla 1. La fase de ingesta estableció la fuente principal, el esquema de almacenamiento y el patrón de evidencias (muestra en CSV y auditoría en texto) que las fases siguientes reutilizan. La fase de preprocesamiento introdujo el motor de procesamiento distribuido y el paradigma ELT, dejando un conjunto limpio como contrato de entrada para la siguiente fase. La fase de enriquecimiento incorporó dos fuentes externas, la lógica de homologación por título y los controles de cardinalidad, y produjo el conjunto analítico final.

Tabla 1

Fases del pipeline, entradas, procesos y salidas principales

Fase	Entrada	Proceso (módulo principal)	Salida principal
Ingesta	API FreeToGame (JSON)	Extracción, validación, persistencia idempotente (src/main.py)	data/database/games.db, data/raw/games.json
Preprocesamiento ELT	games.db (tabla games)	Perfilamiento, limpieza, tipificación y validación con PySpark (src/preprocessing.py)	data/processed/games_cleaned.csv
Enriquecimiento	games_cleaned.csv + APIs GamerPower y MMOBomb (JSON)	Homologación por título, LEFT JOIN, control de cardinalidad (src/enrichment.py)	data/enriched/games_enriched.csv

Nota. Cada fase produce además una muestra en CSV y un reporte de auditoría en texto plano en la carpeta output/.

Arquitectura de la Plataforma

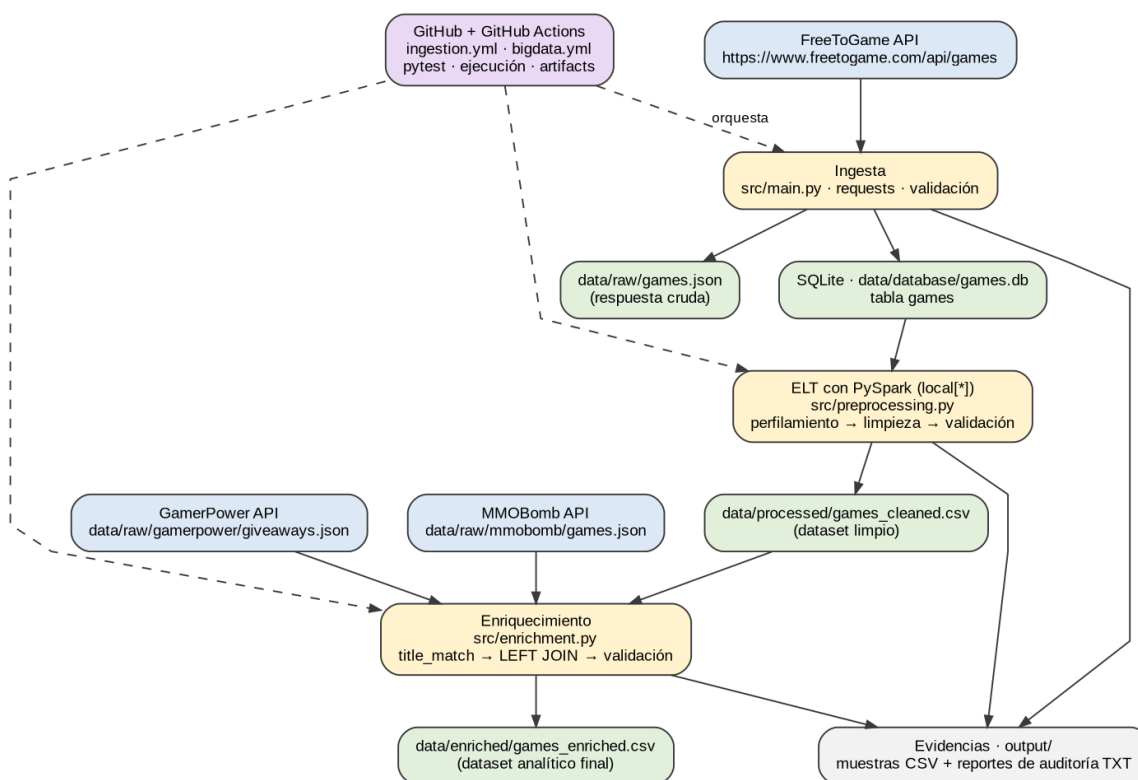
Visión Global

La Figura 1 presenta la arquitectura general. El flujo comienza en la API pública de FreeToGame, cuya respuesta se valida y se persiste dos veces: como archivo JSON crudo (zona

RAW) y como filas de la tabla `games` en la base de datos SQLite (capa de almacenamiento analítico). Desde SQLite, la capa ELT extrae los registros, los carga sin transformar en un DataFrame de Spark y aplica dentro de Spark el perfilamiento, la limpieza y la validación, exportando el resultado a `games_cleaned.csv`. Ese archivo es la base del enriquecimiento, que descarga y conserva en JSON las respuestas de GamerPower y MMOBomb, deriva una clave de homologación por título, ejecuta las uniones en Spark y publica `games_enriched.csv`. Las tres capas escriben evidencias en `output/` y son orquestadas por dos flujos de trabajo de GitHub Actions.

Figura 1

Arquitectura general de la plataforma de datos simulada



Nota. Los nodos azules representan fuentes externas; los amarillos, procesos en Python/PySpark; los verdes, zonas de almacenamiento y conjuntos de datos; el gris, evidencias; el morado, versionamiento y automatización. Las líneas discontinuas indican orquestación. Elaboración propia.

Componentes Principales

La Tabla 2 resume los componentes de la plataforma y su responsabilidad. El diseño separa deliberadamente las responsabilidades por módulo: la configuración (rutas, URLs, constantes, clasificación de campos) está centralizada en `src/config.py`; el acceso a SQLite vive únicamente en `src/database.py` y es reutilizado por la capa ELT en lugar de reimplementarse; la generación de muestras comparte una única función (`write_csv_sample`) para las tres capas; y el ciclo de vida de la sesión de Spark y el puente Pandas–Spark están aislados en `src/spark_session.py`, compartido por el preprocesamiento y el enriquecimiento.

Tabla 2

Componentes de la plataforma y responsabilidad de cada uno

Componente	Rol en la arquitectura	Implementación real
API FreeToGame	Fuente principal: catálogo de videojuegos free-to-play (JSON, sin autenticación).	<code>src/extract.py</code> (petición HTTP, validación, guardado del JSON crudo).
SQLite	Almacenamiento analítico inicial y fuente de la capa ELT.	<code>src/database.py</code> : tabla <code>games</code> , inserción idempotente con <code>INSERT OR REPLACE</code> .
Python	Lenguaje de integración de todas las capas, manejo de errores y códigos de salida.	Módulos en <code>src/</code> , orquestadores <code>main.py</code> , <code>preprocessing.py</code> , <code>enrichment.py</code> .
Requests	Cliente HTTP para las tres APIs.	<code>requests.get()</code> con tiempo de espera de 15 s y captura de excepciones de red.
Pandas	Lectura de SQLite/CSV, puente hacia Spark, homologación de títulos y generación de muestras.	<code>generate_sample.py</code> , <code>title_matching.py</code> , <code>pandas_to_spark()</code> .
PySpark	Motor de procesamiento distribuido (simulado en <code>local[*]</code>) para perfilar, limpiar, validar y unir.	<code>spark_session.py</code> , <code>profiling.py</code> , <code>cleaning.py</code> , <code>validation.py</code> , uniones en <code>enrichment.py</code> .
API GamerPower	Fuente de enriquecimiento: giveaways activos (valor, tipo, estado, fechas).	<code>src/gamerpower_client.py</code> ; RAW en <code>data/raw/gamerpower/giveaways.json</code> .
API MMOBomb	Fuente de enriquecimiento y contraste: catálogo alternativo de juegos.	<code>src/mmobomb_client.py</code> ; RAW en <code>data/raw/mmobomb/games.json</code> .
GitHub	Versionamiento del código, de la configuración y de las evidencias.	Repositorio único con estructura <code>src/</code> , <code>tests/</code> , <code>data/</code> , <code>output/</code> , <code>docs/</code> .

Componente	Rol en la arquitectura	Implementación real
GitHub Actions	Automatización: pruebas, ejecución de las tres capas, verificación y publicación de artifacts.	<code>.github/workflows/ingestion.yml</code> y <code>.github/workflows/bigdata.yml</code> .

Simulación del Entorno Cloud

La plataforma no utiliza servicios de nube reales (AWS, Azure, Google Cloud, Databricks). En su lugar, cada componente local representa una capa que en una arquitectura de nube estaría cubierta por un servicio administrado, como se detalla en la Tabla 3. SQLite cumple el papel de la base de datos analítica de origen; los archivos JSON crudos, el de un almacén de objetos en la zona RAW; PySpark en modo `local[*]` reparte el trabajo entre tantos hilos como núcleos tiene la máquina, empleando el mismo motor de planificación (Catalyst), las mismas APIs de DataFrame y la misma evaluación perezosa que usaría un clúster real; y GitHub Actions actúa como orquestador y ejecutor programado, con ejecuciones diarias definidas por `cron`. La equivalencia es conceptual: permite demostrar el diseño y la lógica de una plataforma distribuida, pero no su comportamiento bajo grandes volúmenes ni su tolerancia a fallos.

Tabla 3

Correspondencia entre los componentes locales y las capas de una arquitectura de nube

Capa conceptual	Componente local (estado actual)	Servicio equivalente en nube (referencia, no implementado)
Zona RAW / data lake	<code>data/raw/*.json</code> versionados en Git	Amazon S3, Azure Blob Storage, Google Cloud Storage
Base de datos analítica	SQLite (<code>data/database/games.db</code>)	Amazon RDS/PostgreSQL, Azure SQL, Cloud SQL
Motor de procesamiento	PySpark <code>local[*]</code> con Apache Arrow	Databricks, Amazon EMR, Google Dataproc
Zona procesada / enriquecida	<code>data/processed/*.csv</code> , <code>data/enriched/*.csv</code>	Tablas Parquet/Delta en almacén de objetos o data warehouse

Capa conceptual	Componente local (estado actual)	Servicio equivalente en nube (referencia, no implementado)
Orquestación y CI/CD	GitHub Actions (push, manual, cron diario)	Airflow, Azure Data Factory, Step Functions, más CI/CD
Evidencias y observabilidad	output/*.txt, output/*.csv, artifacts de 30 días	Registro centralizado, catálogo de datos, monitoreo de calidad

Nota. La columna derecha se presenta únicamente como referencia para una eventual migración; ninguno de esos servicios forma parte de la implementación actual.

Flujo de Datos

Esta sección recorre el pipeline capa por capa. Para cada una se describe la entrada, las operaciones aplicadas tal como están implementadas en el código, las validaciones y las salidas, con su diagrama correspondiente.

Capa de Ingesta

La capa de ingesta (Figura 2) conecta la plataforma con su fuente principal. El orquestador `src/main.py` crea los directorios requeridos, consulta el endpoint `https://www.freetogame.com/api/games` mediante `requests.get()` con un tiempo de espera de 15 segundos y delega la validación al módulo `src/extract.py`, que comprueba en cadena el código de estado HTTP (debe ser 200), que el cuerpo sea JSON válido, que el JSON sea una lista no vacía y que todos sus elementos sean objetos. Cualquier incumplimiento lanza `ExtractionError` y detiene el pipeline con código de salida 1, sin escribir nada en la base de datos. Adicionalmente, el módulo compara las claves del primer registro con los once campos esperados definidos en `config.EXPECTED_FIELDS` y registra advertencias, sin interrumpir, si la API agregó o retiró campos; así se reconoce que la fuente puede cambiar sin que ello rompa la ingesta.

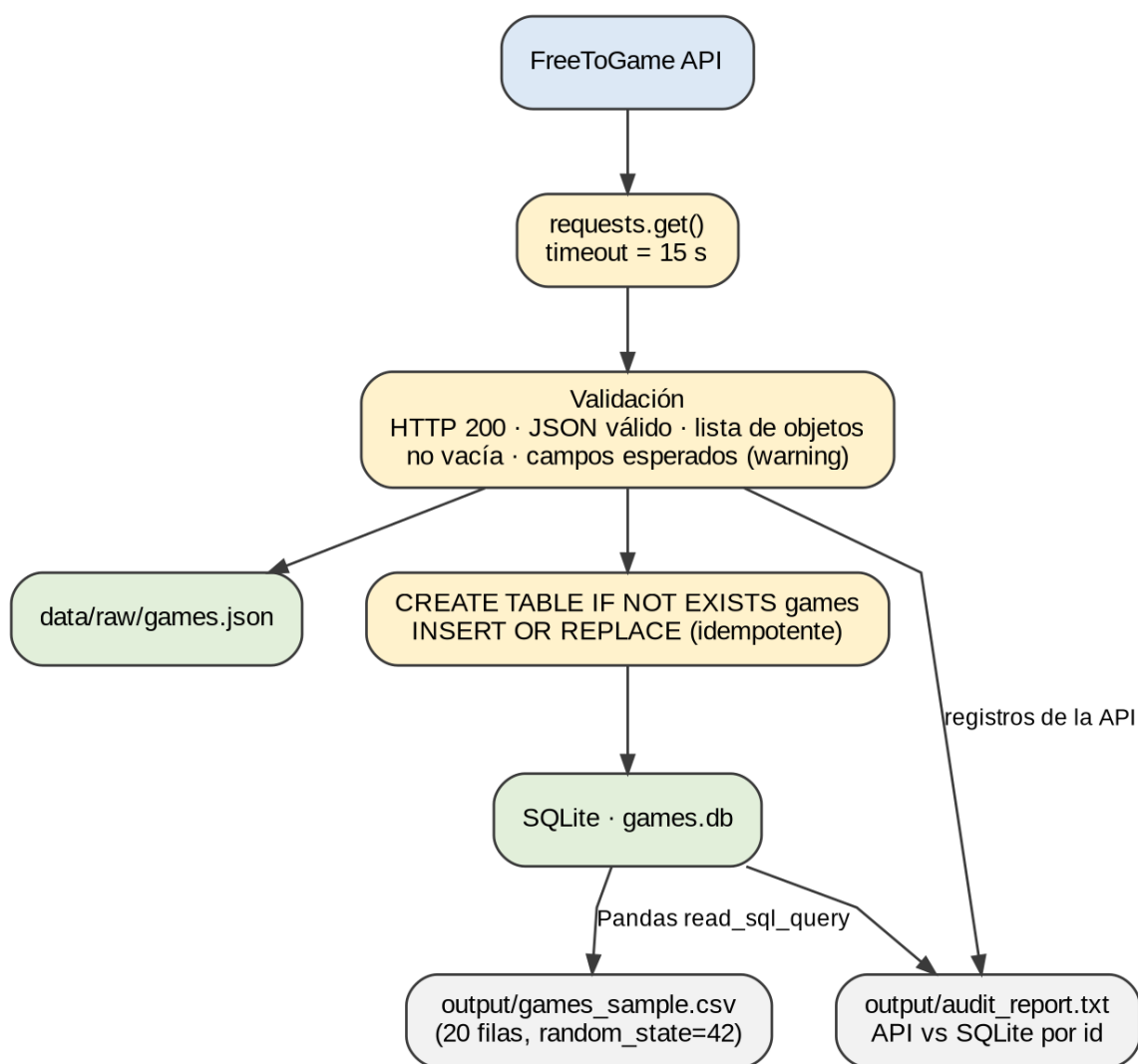
Superada la validación, la respuesta se persiste sin transformar en `data/raw/games.json`, lo que permite auditar exactamente qué entregó la API en cada

ejecución. A continuación, `src/database.py` abre o crea `data/database/games.db`, ejecuta `CREATE TABLE IF NOT EXISTS games` e inserta los registros en una única transacción con `INSERT OR REPLACE`. Esta elección hace que la operación sea idempotente (repetir la ejecución con los mismos datos no genera duplicados) y, al mismo tiempo, que la base refleje siempre el estado más reciente publicado por la API cuando cambia algún atributo de un juego ya almacenado. Los registros sin `id` se omiten y se contabilizan en el registro de ejecución.

Como evidencias, la capa genera con Pandas una muestra reproducible de hasta 20 filas (`random_state = 42`, ordenada por `id`) en `output/games_sample.csv`, leída directamente desde SQLite con `read_sql_query`, y un reporte de auditoría en `output/audit_report.txt` que compara, por `id`, los registros de la API contra los almacenados: conteos, identificadores faltantes o adicionales y diferencias campo a campo (normalizando `None` y espacios para evitar falsos positivos). El resultado final se marca como “VALIDACIÓN EXITOSA” o “VALIDACIÓN CON ERRORES”, y el orquestador devuelve 0 o 2 respectivamente.

Figura 2

Flujo de la capa de ingesta desde la API de FreeToGame hasta SQLite y sus evidencias



Nota. La validación ocurre antes de cualquier escritura; el JSON crudo y la base de datos se persisten en paralelo y la auditoría contrasta ambos lados por `id`. Elaboración propia.

Capa de Preprocesamiento (ELT)

La segunda capa (Figura 3) no vuelve a consultar la API: su única fuente es `games.db`. Sigue el paradigma ELT porque los datos se extraen y se cargan tal cual en el entorno de procesamiento (Spark) y solo después se transforman dentro de él, a diferencia de un ETL clásico en el que la transformación precede a la carga en el destino. Antes de iniciar Spark,

`src/validation.py` verifica que la base exista, que la tabla `games` esté presente y que contenga registros; si no es así, el proceso termina con código 1 y un mensaje que indica ejecutar primero la ingesta.

Extracción y Carga

La extracción reutiliza `database.get_connection` y `database.fetch_all_games`, que seleccionan las once columnas de datos de la tabla (excluyendo la marca interna `ingested_at`). La carga convierte esa lista de registros en un DataFrame de Pandas y luego en un DataFrame de Spark mediante `pandas_to_spark()`, que fuerza `dtype = object` para preservar los valores `None` de Python como `NULL` (evitando que Pandas convierta una columna numérica con ausencias a `float64` con `NaN`, que Spark no trataría como nulo) y usa la conversión vía Apache Arrow habilitada en la sesión. Se justifica el paso intermedio por Pandas porque SQLite carece de un conector JDBC estándar en Spark y el volumen (cientos de registros) no lo convierte en cuello de botella. Tras la carga se valida el esquema contra los campos esperados, reportando columnas faltantes o adicionales.

Perfilamiento

El módulo `src/profiling.py` calcula sobre el DataFrame de Spark, con cifras reales y no estimadas: total de registros y columnas, nulos por columna, cadenas vacías por columna de texto, filas completamente duplicadas, identificadores únicos y filas adicionales por `id` repetido, rango de `id`, validez de `release_date` frente al formato `yyyy-MM-dd` y número de valores distintos en las columnas categóricas `genre`, `platform`, `publisher` y `developer`. Este perfilamiento se ejecuta dos veces, antes y después de la limpieza, para construir la comparación “antes/después” del reporte.

Transformación

Las transformaciones de `src/cleaning.py` se aplican en un orden deliberado, de modo que cada paso prepare el siguiente:

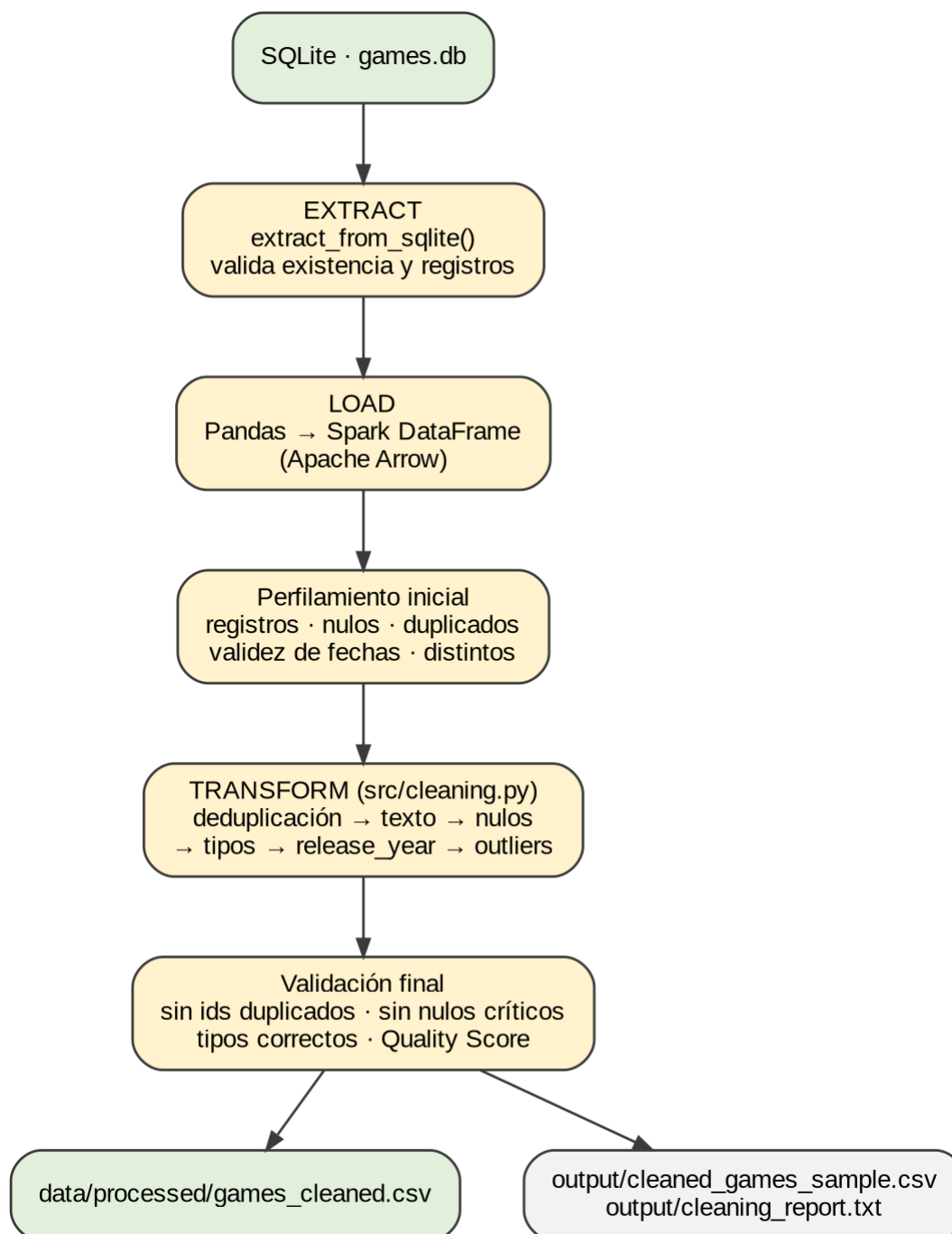
1. Deduplicación en dos niveles: primero filas idénticas en todas sus columnas y luego duplicados por `id`, clave lógica del juego, conservando una única versión por identificador.
2. Normalización de texto en todas las columnas de tipo cadena (incluidas las categóricas): recorte de espacios, colapso de espacios internos múltiples y conversión de cadenas vacías a `NULL`, para que no existan “nulos ocultos”. No se alteran mayúsculas ni se reescriben valores.
3. Tratamiento de nulos diferenciado por importancia del campo: las filas sin `id` o `title` (campos críticos) se eliminan; `genre`, `platform`, `publisher` y `developer` (campos importantes) se imputan con “Unknown”; `thumbnail`, `short_description`, `game_url` y `freetogame_profile_url` (campos descriptivos) se imputan con “Not Available”. Cada imputación queda contabilizada.
4. Corrección de tipos: `id` a `IntegerType`, campos de texto a `StringType` y `release_date` a `DateType` con `to_date(..., 'yyyy-MM-dd')`; los valores no convertibles quedan en `NULL` y se cuentan como invalidados.
5. Columna derivada `release_year`, extraída de la fecha ya validada, para facilitar análisis por año sin volver a parsear la fecha.
6. Análisis de outliers por rango intercuartílico (IQR) sobre las columnas numéricas continuas. En este conjunto la única columna numérica es `id`, un identificador secuencial y no una métrica, por lo que el análisis se documenta explícitamente como no aplicable a `id`, dejando el método implementado para columnas futuras.

Validación Final y Puntaje de Calidad

`validate_clean_dataset()` confirma que no queden `id` duplicados ni nulos en campos críticos y que `id` y `release_date` tengan los tipos esperados; cualquier observación se lista en el reporte y hace que el pipeline devuelva código 2. Se calcula además un *Data Quality Score* de 0 a 100 % como promedio simple de cuatro componentes reproducibles: proporción de registros sin `id` duplicado, proporción sin nulos críticos, corrección de tipos (1,0; 0,5 o 0,0) y proporción de problemas iniciales (duplicados más nulos) resueltos. Las salidas son `data/processed/games_cleaned.csv` (doce columnas: las once originales más `release_year`), `output/cleaned_games_sample.csv` y `output/cleaning_report.txt`; la sesión de Spark se cierra siempre en un bloque `finally`.

Figura 3

Flujo ELT de la capa de preprocesamiento y limpieza con PySpark



Nota. Las etapas Extract y Load preceden a toda transformación, que ocurre dentro de Spark; el perfilamiento se ejecuta antes y después de la limpieza para el reporte comparativo. Elaboración propia.

Capa de Enriquecimiento

La tercera capa (Figura 4) amplía el conjunto limpio con dos APIs públicas conservadas exclusivamente en JSON. `src/gamerpower_client.py` y `src/mmobomb_client.py` siguen el mismo patrón que la ingesta: petición con tiempo de espera, validación de estado HTTP y de JSON, e interpretación tolerante de la estructura (lista plana u objeto con clave contenedora `giveaways` o `games`, respectivamente, porque la forma exacta debe verificarse en tiempo de ejecución). El payload se guarda tal como se recibió en `data/raw/gamerpower/giveaways.json` y `data/raw/mmobomb/games.json`, sin renombrar campos, sin deduplicar y sin normalizar: toda limpieza posterior ocurre solo en memoria.

Homologación por Título

Los tres proveedores tienen sistemas de identificación independientes, por lo que cruzar sus `id` sería incorrecto. La clave lógica de integración es el título del juego, normalizado en una columna auxiliar `title_match` por `src/title_matching.py` sin modificar nunca el campo `title`. La normalización es determinística: elimina el sufijo final “Giveaway”, elimina anotaciones de distribuidor o plataforma entre paréntesis pertenecientes a una lista cerrada (Steam, Epic Games, GOG, Indiegala, Origin, Ubisoft Connect, Humble Bundle, DRM-free, itch.io, Battle.net, PlayStation, Xbox, Switch, entre otras), convierte a minúsculas y recorta y colapsa espacios. Así, “Sausage Hunter (Indiegala) Giveaway” se homologa como “sausage hunter”.

La estrategia de coincidencia tiene dos niveles activos y uno deliberadamente excluido. El primero es la coincidencia exacta sobre `title_match`. El segundo es un mapeo manual explícito en `data/mappings/title_mapping.csv` (columnas `freetogame_title`, `gamerpower_title`, `mmobomb_title`, `match_method`), que se entrega vacío y solo admite

equivalencias verificadas a mano; `apply_title_mapping()` las usa como anulación de `title_match`. El tercero, la coincidencia difusa automática, no se implementa para aceptar cruces: todo título que no coincida exactamente ni esté mapeado queda como no coincidencia, con las columnas de esa fuente en `NULL`, en lugar de forzarse una coincidencia dudosa.

Resolución de Duplicados y Unión

Antes de la unión, `resolve_duplicate_titles()` consolida cada fuente a un registro por `title_match`, con una regla documentada: se ordena por la columna de prioridad de forma descendente (`published_date` en `GamerPower`, `release_date` en `MMOBomb`) y se conserva el más reciente, con desempate por `id` ascendente; los títulos vacíos tras normalizar no participan de la deduplicación ni del cruce. Esta lógica corre en `Pandas`, no como UDF de `Spark`, para evitar la serialización de funciones `Python` hacia la `JVM` que ocasionó errores de recursión en algunos entornos `Windows`. Las columnas seleccionadas de cada fuente se renombran con los prefijos `gamerpower_` y `mmobomb_`, de modo que nunca sobrescriban columnas del conjunto base.

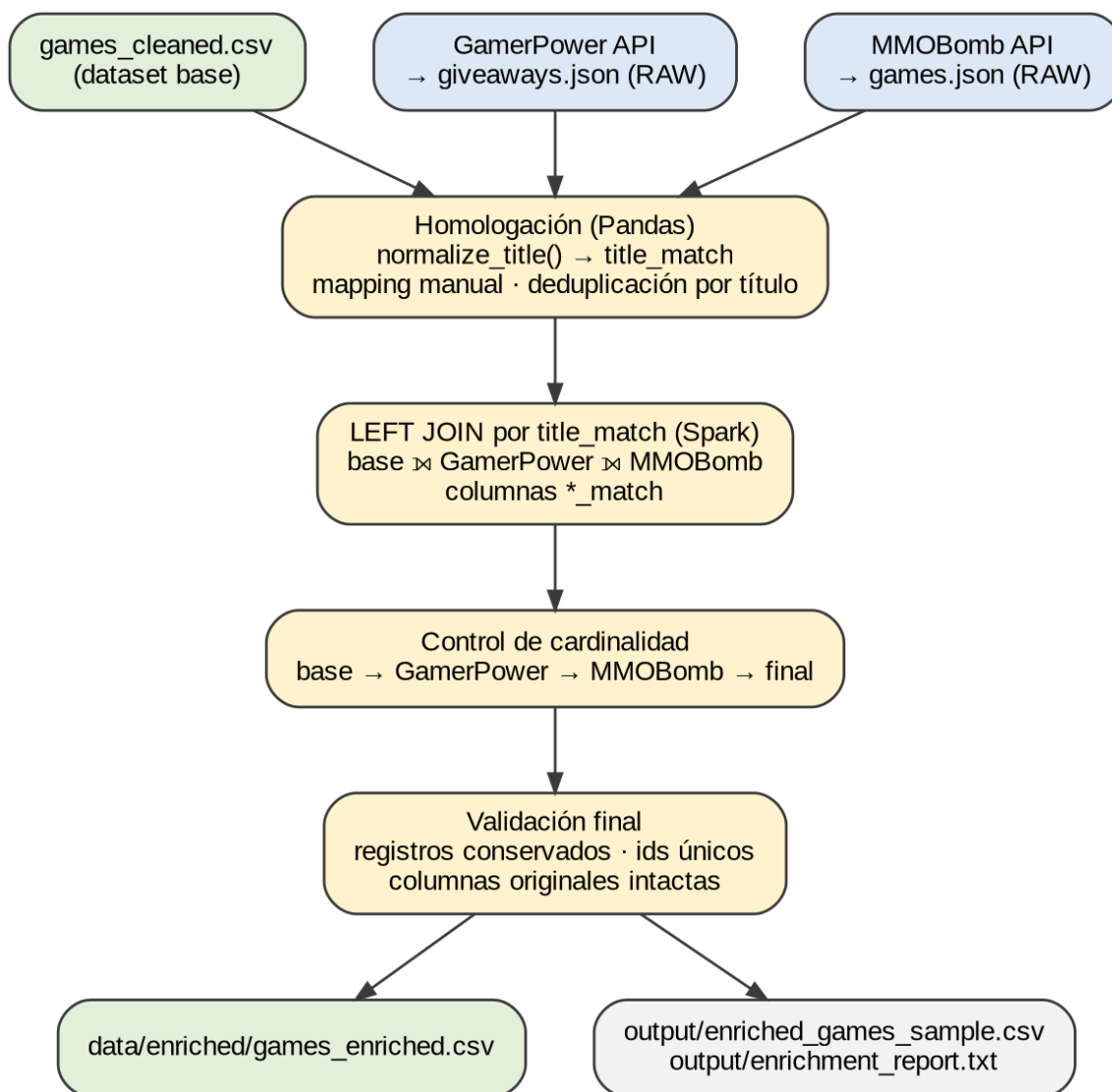
Ya en `Spark`, `left_join_on_title()` ejecuta un `LEFT JOIN` del conjunto base con `GamerPower` y luego con `MMOBomb`. Al haberse dejado cada fuente con `title_match` único, la unión no puede multiplicar filas del lado izquierdo: la granularidad de un juego por registro queda garantizada por construcción y, además, verificada. Para medir la cobertura sin ambigüedad, la clave de la fuente se renombra temporalmente y se cuenta cuántas filas base la tienen no nula tras la unión. Después se agregan las columnas booleanas `genre_match`, `platform_match`, `publisher_match` y `developer_match`, que comparan (sin distinguir mayúsculas ni espacios) los valores de `FreeToGame` contra los de `MMOBomb`; cuando no hubo coincidencia con `MMOBomb` quedan en `NULL` y no en `False`, porque “no se pudo comparar” es distinto de “se comparó y difiere”.

Control de Cardinalidad y Validación Final

En cada etapa se registra la cantidad de registros antes y después (conjunto base, tras GamerPower, tras MMOBomb, final). Al cierre se valida que el conjunto enriquecido tenga las mismas filas que `games_cleaned.csv`, que no existan id duplicados y que las columnas originales no hayan cambiado de valor (mediante `exceptAll` contra el conjunto base). Las salidas son `data/enriched/games_enriched.csv`, `output/enriched_games_sample.csv` y `output/enrichment_report.txt`, este último con cobertura por fuente, notas de homologación, duplicados consolidados, cardinalidad por etapa, validación y una tabla de linaje de las columnas nuevas.

Figura 4

Flujo de la capa de enriquecimiento por título con GamerPower y MMOBomb



Nota. La homologación y la deduplicación ocurren en Pandas antes de crear los DataFrames de Spark; las uniones son externas izquierdas sobre `title_match`, lo que conserva todos los registros del conjunto base. Elaboración propia.

Flujo End-to-End

Visto de extremo a extremo, un registro de videojuego recorre el siguiente camino: (1) es publicado por la API de FreeToGame y descargado en una petición validada; (2) se conserva literalmente en `games.json` y se persiste como fila de `games` en SQLite con su marca de tiempo

de ingesta; (3) es extraído y cargado sin cambios en un DataFrame de Spark; (4) es perfilado, deduplicado, normalizado, imputado y tipificado, y recibe la columna `release_year`; (5) es validado y exportado a `games_cleaned.csv`; (6) recibe la clave derivada `title_match`; (7) se une por la izquierda con el registro más reciente de GamerPower y con el de MMOBomb que compartan esa clave, si existen, y recibe las columnas de comparación; (8) es verificado en cardinalidad, unicidad y consistencia; y (9) se publica en `games_enriched.csv`, con su procedencia documentada en los tres reportes de auditoría. Cada uno de estos pasos deja un artefacto verificable, y cualquier fallo en la cadena detiene la ejecución con un código de salida distinto de cero.

Automatización y Control de Versiones

GitHub como Repositorio Único

Todo el proyecto reside en un único repositorio de GitHub que versiona código, configuración, pruebas, flujos de trabajo y también las evidencias generadas. La política de exclusión (`.gitignore`) omite entornos virtuales, cachés y metadatos locales de Spark, pero a propósito no ignora `games.json`, `games.db`, `games_cleaned.csv`, los JSON crudos de GamerPower y MMOBomb, `title_mapping.csv`, `games_enriched.csv` ni los archivos de `output/`, porque constituyen la evidencia “viva” del último estado del pipeline. Esta decisión favorece la trazabilidad, aunque implica versionar archivos de datos, cuestión que se retoma en las limitaciones.

Flujos de Trabajo de GitHub Actions

La automatización se divide en dos flujos independientes. `ingestion.yml` ejecuta la capa de ingesta; `bigdata.yml` ejecuta en un mismo trabajo la capa ELT y, a continuación, la capa de

enriquecimiento, y depende de que `games.db` ya exista en el repositorio (si no existe, falla explícitamente indicando que debe ejecutarse primero la ingesta). Ambos se disparan en cada push, pueden lanzarse manualmente (`workflow_dispatch`) y tienen una ejecución programada diaria: la ingesta a las 06:00 UTC y el flujo de procesamiento a las 06:30 UTC, de modo que el segundo corra después del primero. La Tabla 4 detalla los pasos reales de cada flujo.

Tabla 4

Pasos de los flujos de trabajo de GitHub Actions

Flujo	Pasos ejecutados (en orden)	Artifacts publicados (retención 30 días)
ingestion.yml (EA1 – Ingestión de Datos desde API)	1. Checkout. 2. Python 3.11 con caché de pip. 3. <code>pip install -r requirements.txt</code> . 4. <code>pytest -v. 5. python src/main.py</code> . 6. Verificación de existencia de <code>games.db</code> , <code>games_sample.csv</code> y <code>audit_report.txt</code> . 7. Impresión del contenido de las evidencias en el log. 8. Publicación de artifacts.	evidencias-ingestion-ea1: <code>games.json</code> , <code>games.db</code> , <code>games_sample.csv</code> , <code>audit_report.txt</code> .
bigdata.yml (EA2 + EA3 – ELT, Limpieza y Enriquecimiento)	1. Checkout. 2. Python 3.11. 3. Java 17 (Temurin), requerido por la JVM de Spark. 4. Dependencias. 5. Verificación de <code>games.db</code> . 6. <code>pytest -v</code> con <code>PYSPARK_PYTHON=python</code> y <code>SPARK_LOCAL_IP=127.0.0.1</code> . 7. <code>python src/preprocessing.py</code> . 8–9. Verificación e impresión de las evidencias ELT. 10. Artifact ELT. 11. <code>python src/enrichment.py</code> . 12. Verificación de los JSON crudos. 13–14. Verificación e impresión de las evidencias de enriquecimiento. 15–18. Artifacts.	evidencias-preprocesamiento-ea2; <code>gamerpower_raw</code> ; <code>mmobomb_raw</code> ; <code>enriched_dataset</code> ; <code>enrichment_evidence</code> (muestra + reporte).

Nota. Las variables `PYSPARK_PYTHON` y `SPARK_LOCAL_IP` garantizan que el controlador y los ejecutores usen el mismo intérprete y que Spark no intente resolver el nombre de host del ejecutor de CI por red.

Pruebas Automatizadas

Antes de ejecutar cualquier pipeline, ambos flujos corren la suite de `pytest`, compuesta por tres módulos que suman 40 pruebas: `tests/test_ingestion.py` (12 pruebas: estructura válida e inválida de la API, código HTTP incorrecto, guardado del JSON, creación de base y tabla, inserción e idempotencia, generación de muestra y auditoría, y detección de registros faltantes y de diferencias de campo), `tests/test_preprocessing.py` (15 pruebas: extracción desde

SQLite y sus errores, creación del DataFrame, detección y eliminación de duplicados, nulos, tipos, año derivado, outliers, esquema, pipeline completo, muestra y reporte) y `tests/test_enrichment.py` (13 pruebas: descarga y archivo crudo de ambas APIs, estructuras inválidas, carga del conjunto base, normalización de títulos, deduplicación por recencia, LEFT JOIN con cobertura, unión de dos fuentes sin duplicados y con columnas `*_match`, y generación de CSV, muestra y reporte). Todas las llamadas HTTP se simulan con `unittest.mock`, de modo que la suite no depende de la red, y las pruebas de Spark usan DataFrames pequeños contruidos para cada caso, compartiendo una única sesión por módulo. El archivo `conftest.py` agrega la raíz del proyecto a `sys.path` para resolver los imports `src.*`.

Evidencias y Artifacts

El proyecto combina dos mecanismos complementarios de evidencia. Los archivos versionados en el repositorio muestran el último estado confirmado, visible para cualquiera que lo clone. Los artifacts de GitHub Actions, en cambio, son la evidencia oficial de cada ejecución individual: se generan sin modificar el repositorio, se descargan desde la página de la ejecución y permiten comparar resultados entre corridas, por ejemplo para observar cómo cambia el catálogo o la cobertura de giveaways con el tiempo. Cada paso de verificación usa `test -f` sobre los archivos esperados y `if-no-files-found: error` en la publicación, por lo que la ausencia de una evidencia hace fallar el flujo de forma explícita.

Modelo de Datos

El modelo de datos de la plataforma tiene una particularidad que conviene explicitar desde el inicio: existe un único modelo físico relacional, la tabla `games` en SQLite, y dos conjuntos de datos derivados en formato CSV que no son tablas de base de datos, sino resultados materializados

del procesamiento. Confundirlos llevaría a documentar relaciones o claves foráneas que no existen físicamente. Esta sección describe cada nivel por separado y luego el linaje que los conecta.

Modelo Físico: Base de Datos SQLite

La base `data/database/games.db` contiene una sola tabla, `games`, cuyo esquema se define en `src/database.py` y se resume en la Tabla 5. El esquema refleja directamente los campos que entrega la API de FreeToGame, sin inventar atributos adicionales, y agrega una única columna interna de control, `ingested_at`, con la fecha y hora de inserción. El identificador `id` de la API se adopta como clave primaria natural porque es único y estable por juego. `release_date` se almacena como texto en formato ISO (YYYY-MM-DD), tal como llega de la API, y su conversión a tipo fecha se difiere a la capa ELT, donde puede validarse y contabilizarse. No existen índices adicionales, claves foráneas ni otras tablas.

Tabla 5

Esquema real de la tabla games en SQLite

Campo	Tipo	PK	Nulo	Descripción
<code>id</code>	INTEGER	Sí	No	Identificador único del juego asignado por FreeToGame.
<code>title</code>	TEXT	No	No (NOT NULL)	Título del videojuego.
<code>thumbnail</code>	TEXT	No	Sí	URL de la imagen en miniatura.
<code>short_description</code>	TEXT	No	Sí	Descripción breve.
<code>game_url</code>	TEXT	No	Sí	URL para acceder al juego.
<code>genre</code>	TEXT	No	Sí	Género (p. ej., MMORPG, Shooter).
<code>platform</code>	TEXT	No	Sí	Plataforma (p. ej., PC (Windows), Web Browser).
<code>publisher</code>	TEXT	No	Sí	Empresa distribuidora.
<code>developer</code>	TEXT	No	Sí	Estudio desarrollador.
<code>release_date</code>	TEXT	No	Sí	Fecha de lanzamiento en formato ISO YYYY-MM-DD.

Campo	Tipo	PK	Nulo	Descripción
freetogame_profile_url	TEXT	No	Sí	URL del perfil del juego en FreeToGame.
ingested_at	TEXT	No	No (DEFAULT datetime('now'))	Marca de tiempo interna de la inserción; no proviene de la API.

Nota. Tipos tomados literalmente de la sentencia `CREATE TABLE` en `src/database.py`. No hay claves foráneas: la tabla es la única de la base.

Modelo Físico, Conjunto Procesado y Conjunto Enriquecido

Es importante distinguir cinco conceptos que conviven en el proyecto. El *modelo físico* es la tabla `games` recién descrita. El *DataFrame de Spark* es la representación distribuida y transitoria de los datos dentro de una sesión de procesamiento; no persiste al terminar la ejecución. El *conjunto procesado* (`data/processed/games_cleaned.csv`) es la materialización del *DataFrame* ya limpio: contiene las once columnas de datos de `games` (sin `ingested_at`) más la derivada `release_year`; sus tipos en Spark eran `IntegerType`, `StringType` y `DateType`, pero en CSV se serializan como texto, por lo que cualquier consumidor debe volver a tipificarlos al cargarlos. El *conjunto enriquecido* (`data/enriched/games_enriched.csv`) es la materialización del resultado de las uniones. El *modelo lógico* es la descripción conceptual de cómo se relacionan estos conjuntos, que se detalla a continuación y no está implementada como esquema relacional.

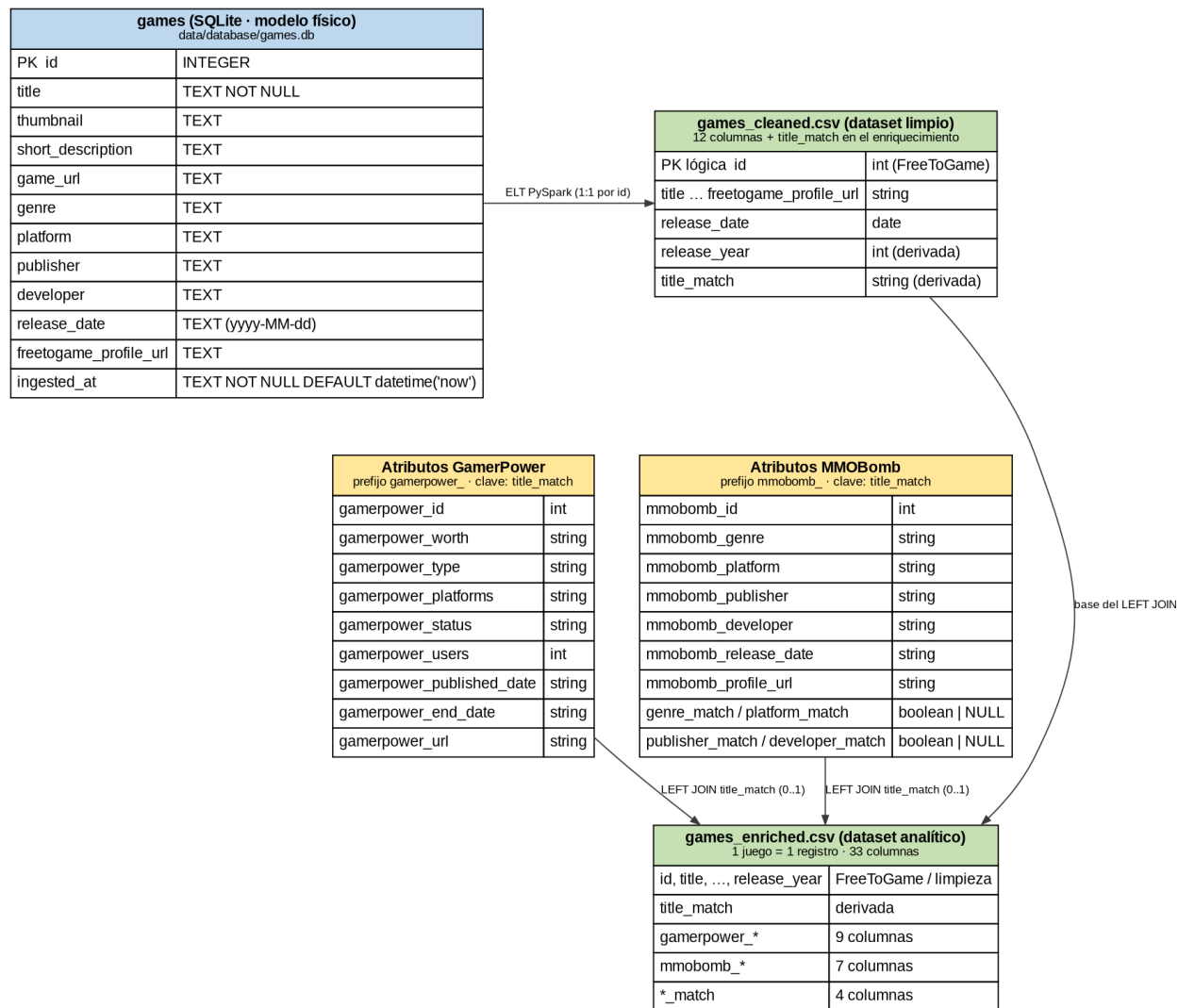
Diagrama del Modelo de Datos

La Figura 5 integra los niveles anteriores. A la izquierda aparece la única entidad física (`games`); en el centro, el conjunto limpio y los bloques de atributos que aportan `GamerPower` y `MMOBomb`; a la derecha, el conjunto enriquecido. Las flechas no son claves foráneas: representan derivaciones (ELT, uno a uno por `id`) y uniones externas izquierdas por `title_match` con cardinalidad cero o uno del lado de cada fuente, garantizada por la deduplicación previa. El

enriquecimiento es, por tanto, un proceso de integración de fuentes externas sobre un conjunto de datos, y no una transformación de la base SQLite en un modelo relacional con nuevas tablas.

Figura 5

Modelo físico (tabla games) y modelo lógico de los conjuntos derivados



Nota. Solo la tabla `games` existe como estructura relacional. Los demás bloques describen columnas de archivos CSV y las relaciones indican derivación o unión, no restricciones de integridad referencial. Elaboración propia.

Modelo Lógico del Conjunto Enriquecido

El conjunto enriquecido tiene granularidad de un juego por registro, con `id` de FreeToGame como clave principal lógica y `title_match` como clave auxiliar de integración.

Consta de 33 columnas organizadas en cuatro grupos: los doce atributos del conjunto limpio, la clave derivada `title_match`, nueve atributos de GamerPower, siete de MMOBomb y cuatro columnas de comparación. Las Tablas 6, 7 y 8 describen cada grupo con su origen. Los identificadores `gamerpower_id` y `mmobomb_id` se conservan como columnas de procedencia y nunca reemplazan a `id`.

Tabla 6

Atributos base y clave auxiliar del conjunto enriquecido

Columna	Tipo lógico	Origen	Observación
<code>id</code>	entero	FreeToGame (ingesta)	Clave principal lógica; única por construcción.
<code>title</code> , <code>thumbnail</code> , <code>short_description</code> , <code>game_url</code> , <code>genre</code> , <code>platform</code> , <code>publisher</code> , <code>developer</code> , <code>freetogame_profile_url</code>	cadena	FreeToGame (ingesta), normalizados e imputados en la capa ELT	Espacios normalizados; nulos imputados con “Unknown” o “Not Available” según el campo.
<code>release_date</code>	fecha (ISO en CSV)	FreeToGame, tipificada en la capa ELT	NULL si el valor original no era una fecha válida.
<code>release_year</code>	entero	Derivada de <code>release_date</code> (capa ELT)	Año de lanzamiento.
<code>title_match</code>	cadena	Derivada de <code>title</code> (capa de enriquecimiento)	Clave de homologación; <code>title</code> nunca se modifica.

Tabla 7

Atributos incorporados desde GamerPower (prefijo `gamerpower_`)

Columna	Campo de origen en la API	Contenido
<code>gamerpower_id</code>	<code>id</code>	Identificador del giveaway en GamerPower (procedencia).
<code>gamerpower_worth</code>	<code>worth</code>	Valor estimado del giveaway.
<code>gamerpower_type</code>	<code>type</code>	Tipo de promoción (p. ej., <code>game</code> , <code>loot</code> , <code>beta</code>).
<code>gamerpower_platforms</code>	<code>platforms</code>	Plataformas en las que aplica.

Columna	Campo de origen en la API	Contenido
gamerpower_status	status	Estado de la promoción.
gamerpower_users	users	Usuarios que han reclamado la promoción.
gamerpower_published_date	published_date	Fecha de publicación; criterio de recencia en la deduplicación.
gamerpower_end_date	end_date	Fecha de expiración.
gamerpower_url	gamerpower_url	URL del giveaway.

Nota. Se seleccionan solo los campos que aportan valor analítico; se descartan `description`, `instructions`, `image`, `thumbnail` y `open_giveaway_url`. Todas las columnas quedan en NULL cuando el juego no tiene giveaway coincidente.

Tabla 8

Atributos incorporados desde MMOBomb (prefijo `mmobomb_`) y columnas de comparación

Columna	Campo de origen en la API	Contenido
mmobomb_id	id	Identificador del juego en MMOBomb (procedencia).
mmobomb_genre	genre	Género según MMOBomb.
mmobomb_platform	platform	Plataforma según MMOBomb.
mmobomb_publisher	publisher	Distribuidor según MMOBomb.
mmobomb_developer	developer	Desarrollador según MMOBomb.
mmobomb_release_date	release_date	Fecha de lanzamiento según MMOBomb; criterio de recencia en la deduplicación.
mmobomb_profile_url	profile_url	URL del perfil en MMOBomb.
genre_match, platform_match, publisher_match, developer_match	Comparación FreeToGame vs. MMOBomb	True si coinciden (sin distinguir mayúsculas ni espacios), False si difieren, NULL si no hubo coincidencia de título.

Claves y Relaciones

En el nivel físico, la única clave es la primaria `id` de `games`. En el nivel lógico, `id` se propaga sin cambios como clave principal de los conjuntos procesado y enriquecido, y la relación entre `games` y `games_cleaned.csv` es uno a uno por `id` (salvo las filas eliminadas por duplicidad o por nulos críticos). La relación entre el conjunto limpio y cada fuente de

enriquecimiento es de uno a cero-o-uno sobre `title_match`: cada juego base se asocia como máximo con un registro de GamerPower y con uno de MMOBomb, porque ambas fuentes se dejan con `title_match` único antes de la unión. Varios juegos base podrían compartir el mismo `title_match` (por ejemplo, reediciones); en ese caso cada uno se integra de forma independiente por su propio `id`, y la situación se anota en el reporte de enriquecimiento. No hay integridad referencial impuesta por el motor de base de datos; la consistencia se asegura mediante las validaciones del pipeline.

Linaje de Datos

El linaje describe cómo cada conjunto se deriva del anterior y de dónde proviene cada columna (Tabla 9). Esta información no es solo documental: el pipeline la genera automáticamente en la última sección de `output/enrichment_report.txt` a partir del diccionario `DATA_LINEAGE_EA3` de `src/config.py`, y las columnas nuevas llevan prefijos que hacen evidente su procedencia en el propio archivo.

Tabla 9

Linaje de los conjuntos de datos de la plataforma

Conjunto o archivo	Fuente	Capa	Propósito
<code>data/raw/games.json</code>	API FreeToGame	Ingesta	Respuesta cruda, evidencia literal de lo recibido.
<code>data/database/games.db</code> (tabla <code>games</code>)	<code>games.json</code> validado	Ingesta	Almacenamiento analítico idempotente; fuente de la capa ELT.
<code>data/processed/games_cleaned.csv</code>	SQLite + PySpark	Preprocesamiento	Conjunto limpio y tipificado; base del enriquecimiento.
<code>data/raw/gamerpower/giveaways.json</code>	API GamerPower	Enriquecimiento	Respuesta cruda de giveaways, sin transformar.

Conjunto o archivo	Fuente	Capa	Propósito
data/raw/mmobomb/games.json	API MMOBomb	Enriquecimiento	Respuesta cruda del catálogo alternativo, sin transformar.
data/mappings/title_mapping.csv	Configuración manual	Enriquecimiento	Equivalencias de título verificadas a mano (vacío por defecto).
data/enriched/games_enriched.csv	Conjunto limpio + GamerPower + MMOBomb	Enriquecimiento	Conjunto analítico final, un registro por juego.

Justificación del Modelo de Datos

La cadena FreeToGame → SQLite → games_cleaned.csv → games_enriched.csv es apropiada para el propósito de la plataforma por varias razones. Primero, favorece la trazabilidad: cada etapa deja una materialización propia y un reporte con cifras reales, de modo que cualquier valor del conjunto final puede rastrearse hasta la respuesta cruda de la API que lo originó. Segundo, es simple: un único modelo físico con una tabla evita sobrediseñar un esquema relacional para datos que provienen de una sola entidad de negocio, mientras que el enriquecimiento se resuelve como integración de conjuntos sin introducir claves foráneas artificiales entre proveedores que no comparten identificadores. Tercero, separa responsabilidades por etapa, lo que permite reejecutar solo la capa afectada cuando cambia una fuente. Cuarto, es reproducible: la inserción es idempotente, las muestras usan semilla fija y todas las reglas (normalización, imputación, deduplicación, recencia) son determinísticas. Quinto, facilita la auditoría, porque la granularidad de un juego por registro se garantiza por construcción y se verifica al final. Finalmente, deja el terreno preparado para el análisis: el conjunto enriquecido ya contiene la clave, los atributos originales intactos, los atributos externos con prefijo y las columnas de comparación, listos para agregaciones por género, plataforma, año o actividad promocional.

Como observación de transparencia frente al enunciado académico, que menciona “tablas involucradas” y “relaciones entre las tablas”, el proyecto no cuenta con múltiples tablas relacionadas por claves foráneas. La representación lógica de la Figura 5 cumple la función de mostrar las interconexiones reales. Una evolución natural, tratada en las recomendaciones, sería materializar el conjunto enriquecido en un modelo dimensional con una tabla de hechos de promociones y dimensiones de juego, plataforma y proveedor.

Herramientas y Tecnologías

La Tabla 10 justifica cada herramienta a partir de su uso real en el repositorio. Todas son de código abierto y están declaradas en `requirements.txt` con rangos de versión acotados (`requests`, `pandas`, `pytest`, `pyspark 3.5.x`, `pyarrow`, `setuptools`), además de `sqlite3` y `json` de la biblioteca estándar y de un JDK 11 o 17 para la JVM de Spark.

Tabla 10

Justificación de las herramientas empleadas

Herramienta	Uso en el proyecto	Justificación
Python 3.10+	Lenguaje de todas las capas; orquestadores con códigos de salida 0/1/2; excepciones específicas por módulo.	Ecosistema maduro para ingeniería de datos; integra de forma natural HTTP, SQLite, Pandas y Spark; legible y testeable.
Requests	Consumo de las tres APIs con tiempo de espera de 15 s y manejo de <code>Timeout</code> , <code>ConnectionError</code> y <code>RequestException</code> .	Cliente HTTP de facto en Python; permite controlar errores de red de forma explícita, condición necesaria para un pipeline automatizado.
SQLite	Base analítica de la ingesta y fuente de la capa ELT.	Sin servidor, portable, incluida en Python y versionable en Git; suficiente para cientos de registros y adecuada para simular la capa de almacenamiento.
Pandas	Lectura de SQLite y CSV, puente hacia Spark, homologación y deduplicación de títulos, muestras reproducibles.	Ideal para tareas por lotes pequeñas y para la generación de evidencias; evita UDFs de Spark en la lógica de títulos, con lo que se eluden problemas de serialización.

Herramienta	Uso en el proyecto	Justificación
PySpark 3.5	Perfilamiento, limpieza, tipificación, validación y uniones sobre DataFrames distribuidos en <code>local[*]</code> .	Aporta el paradigma de procesamiento distribuido y una API que se ejecuta sin cambios en un clúster real; ver subsección específica.
Apache Arrow (pyarrow)	Conversión columnar Pandas ↔ Spark habilitada en la sesión.	Reemplaza la ruta antigua basada en RDD y <code>cloudpickle</code> , que fallaba con <code>RecursionError</code> en ciertos entornos Windows; además es más rápida.
pytest	40 pruebas unitarias con mocks HTTP y DataFrames de Spark pequeños.	Garantiza que cada capa se valide antes de ejecutarse en CI y documenta el comportamiento esperado.
GitHub	Repositorio único de código, configuración, pruebas y evidencias.	Trazabilidad histórica de código y datos; base para la colaboración y para la automatización.
GitHub Actions	Dos flujos con disparo por <code>push</code> , manual y programado; verificación y artifacts.	Orquestación sin infraestructura propia; deja evidencia oficial de cada ejecución mediante logs y artifacts descargables.

Justificación de PySpark en un Entorno Local

Podría objetarse que un conjunto de cientos de registros no requiere Spark. La elección se justifica por el objetivo de la plataforma, que es demostrar el diseño de un pipeline de Big Data y no solo procesar el volumen actual. Al expresar el perfilamiento, la limpieza y las uniones como transformaciones sobre DataFrames de Spark, el código queda escrito contra la misma API que ejecutaría un clúster administrado: bastaría cambiar el `master` de `local[*]` a la URL del clúster y sustituir las rutas locales por rutas de almacenamiento distribuido. El modo `local[*]` conserva las particiones, el paralelismo por hilos, la planificación con Catalyst y la evaluación perezosa, por lo que la simulación es fiel a la lógica de ejecución distribuida. La sesión se configura con cuatro particiones de *shuffle* (un número mayor solo agregaría sobrecarga con datos pequeños), nivel de registro `WARN` y conversión vía Arrow con `fallback` deshabilitado para que cualquier problema sea explícito. Se aplica además el workaround oficial de importar `setuptools` antes de Spark para proveer `distutils` en Python 3.12+ (Apache Software Foundation, s.f.-b).

La limitación correspondiente debe declararse con claridad: el entorno actual utiliza ejecución local en una sola máquina y, por tanto, no representa una infraestructura Big Data distribuida real ni permite medir escalabilidad, tolerancia a fallos o costos.

Calidad y Validación

La calidad de los datos se controla en las tres capas mediante mecanismos que quedan registrados en los reportes de `output/`, como se resume en la Tabla 11. El principio común es que ninguna cifra de los reportes se estima ni se fija a mano: todas se calculan durante la ejecución sobre los datos reales, y cualquier observación hace que el orquestador devuelva el código de salida 2 para que el flujo de CI la haga visible.

Tabla 11

Mecanismos de calidad y validación por capa

Capa	Controles implementados	Evidencia
Ingesta	Código HTTP 200; JSON válido; lista no vacía de objetos; advertencia por campos faltantes o adicionales; omisión de registros sin <code>id</code> ; auditoría API vs. SQLite por <code>id</code> (faltantes, adicionales, diferencias campo a campo).	<code>output/audit_report.txt</code> con resultado “VALIDACIÓN EXITOSA” o “CON ERRORES”.
Preprocesamiento	Pre-verificación de base, tabla y registros; validación de esquema; perfilamiento antes/después; deduplicación completa y por <code>id</code> ; nulos por criticidad; tipificación con conteo de fechas invalidadas; validación final (sin <code>id</code> duplicados, sin nulos críticos, tipos correctos); Data Quality Score de cuatro componentes.	<code>output/cleaning_report.txt</code> con secciones de extracción, perfilamiento, operaciones, antes/después, validación y resultado.
Enriquecimiento	Pre-verificación del conjunto base; validación HTTP/JSON de ambas APIs; conservación literal de los RAW; homologación determinística; mapeo manual explícito; deduplicación por recencia; cobertura por fuente; control de cardinalidad por etapa; unicidad de <code>id</code> ; consistencia de columnas originales (<code>exceptAll</code>); columnas <code>*_match</code> con NULL cuando no es comparable.	<code>output/enrichment_report.txt</code> con fuentes, cobertura, homologación, duplicados, cardinalidad, validación, resultado y linaje.

Seguridad y Configuración

Las tres APIs consumidas son públicas y no requieren autenticación, por lo que el proyecto no gestiona credenciales ni secretos y no incluye un mecanismo de gestión de secretos; sería necesario incorporarlo (por ejemplo, GitHub Secrets o un gestor de la nube) si en el futuro se integrara una fuente autenticada. Los aspectos de robustez implementados son: tiempos de espera explícitos en todas las peticiones; captura diferenciada de errores de red, de código HTTP y de JSON con excepciones propias por módulo; validación de estructura antes de cualquier escritura; conservación de las respuestas crudas para auditoría; dependencias con rangos de versión acotados; y configuración centralizada en `src/config.py` (rutas construidas con `pathlib`, URLs, constantes de muestreo, clasificación de campos y linaje), que evita valores mágicos dispersos y facilita adaptar el proyecto a otro entorno. Toda la ejecución en CI usa versiones fijas de Python (3.11) y Java (17) para garantizar reproducibilidad.

Evidencias Generadas

La Tabla 12 documenta los artefactos que produce el pipeline y su papel en la trazabilidad. Las cifras concretas de cada ejecución (registros ingeridos, duplicados eliminados, nulos imputados, cobertura por fuente, puntaje de calidad) se leen directamente de los reportes correspondientes; la Tabla 13 se deja como plantilla editable para consignarlas a partir de la última ejecución del repositorio, evitando transcribir valores que podrían haber cambiado, dado que las APIs se consultan diariamente.

Tabla 12*Artefactos generados por el pipeline y su función en la trazabilidad*

Artefacto	Capa	Contenido	Función
data/raw/games.json	Ingesta	Respuesta literal de FreeToGame.	Permite auditar qué se recibió en cada ejecución.
data/database/games.db	Ingesta	Tabla games poblada.	Almacenamiento analítico; fuente de la capa ELT.
output/games_sample.csv	Ingesta	Hasta 20 filas, semilla 42, leídas de SQLite.	Muestra representativa de lo almacenado.
output/audit_report.txt	Ingesta	Comparación API vs. SQLite por id.	Confirma integridad de la ingesta.
data/processed/games_cleaned.csv	Preprocesamiento	Conjunto limpio completo (12 columnas).	Contrato de entrada del enriquecimiento.
output/cleaned_games_sample.csv	Preprocesamiento	Muestra reproducible del conjunto limpio.	Evidencia del resultado de la limpieza.
output/cleaning_report.txt	Preprocesamiento	Perfilamiento inicial y final, operaciones, antes/después, validación, Quality Score.	Auditoría del proceso ELT.
data/raw/gamerpower/giveaways.json	Enriquecimiento	Respuesta literal de GamerPower.	Procedencia de las columnas gamerpower_*.
data/raw/mmobomb/games.json	Enriquecimiento	Respuesta literal de MMOBomb.	Procedencia de las columnas mmobomb_*.
data/enriched/games_enriched.csv	Enriquecimiento	Conjunto analítico final (33 columnas).	Salida final de la plataforma.
output/enriched_games_sample.csv	Enriquecimiento	Muestra reproducible del conjunto enriquecido.	Evidencia de columnas originales y nuevas.
output/enrichment_report.txt	Enriquecimiento	Cobertura, homologación, duplicados, cardinalidad, validación, linaje.	Auditoría de la integración.

Tabla 13

Plantilla de métricas de la última ejecución (completar desde los reportes reales)

Métrica	Valor	Fuente en el repositorio
Registros extraídos desde la API de FreeToGame	[completar]	output/audit_report.txt
Registros almacenados en SQLite	[completar]	output/audit_report.txt
Resultado de la auditoría de ingesta	[completar]	output/audit_report.txt
Registros antes / después de la limpieza	[completar] / [completar]	output/cleaning_report.txt
Duplicados eliminados (completos / por id)	[completar] / [completar]	output/cleaning_report.txt
Valores nulos antes / después	[completar] / [completar]	output/cleaning_report.txt
Fechas invalidadas por formato	[completar]	output/cleaning_report.txt
Data Quality Score	[completar] %	output/cleaning_report.txt
Registros descargados de GamerPower / MMOBomb	[completar] / [completar]	output/enrichment_report.txt
Cobertura GamerPower / MMOBomb	[completar] % / [completar] %	output/enrichment_report.txt
Registros base / finales del conjunto enriquecido	[completar] / [completar]	output/enrichment_report.txt
Resultado del enriquecimiento	[completar]	output/enrichment_report.txt

Nota. Los valores se dejan en blanco de forma deliberada para no consignar cifras que no correspondan a la ejecución vigente del repositorio.

Limitaciones

La plataforma cumple su objetivo de simulación, pero presenta limitaciones que deben tenerse presentes antes de extrapolar sus resultados o de llevarla a producción:

- SQLite es una base local de un solo archivo, sin concurrencia de escritura ni administración remota; no equivale a una base analítica administrada.
- PySpark se ejecuta en modo local en una sola máquina; no se evalúa escalabilidad horizontal, tolerancia a fallos ni costo de cómputo.

- Los conjuntos procesado y enriquecido se materializan en CSV, un formato de texto sin esquema ni compresión que pierde los tipos (fechas y enteros se serializan como texto) y no es óptimo para grandes volúmenes.
- Las tres APIs externas pueden cambiar de estructura o de disponibilidad; el pipeline advierte y falla de forma controlada, pero no se adapta automáticamente a cambios de esquema.
- La homologación por título exacto (tras normalizar) es conservadora por diseño: puede producir no coincidencias para títulos con variantes ortográficas, subtítulos o ediciones, que solo se resuelven agregando equivalencias manuales.
- GamerPower describe promociones vigentes en el momento de la consulta; las columnas `gamerpower_*` son una fotografía temporal y no un histórico de promociones.
- Las evidencias de datos se versionan en Git, lo que es adecuado para el volumen actual pero no escala para archivos grandes ni para históricos.
- GitHub Actions orquesta ejecuciones programadas, pero no reemplaza un orquestador de datos con dependencias entre tareas, reintentos granulares, monitoreo y alertas; el flujo de procesamiento depende de que la ingesta se haya confirmado previamente en el repositorio.
- No existe un catálogo de datos ni control de calidad continuo más allá de los reportes por ejecución.

Recomendaciones y Evolución hacia la Nube

Las siguientes recomendaciones describen una arquitectura futura propuesta; ninguna forma parte de la implementación actual. La Tabla 14 contrasta el estado presente con la evolución sugerida por capa, y a continuación se desarrollan las prioridades.

Tabla 14*Estado actual frente a la arquitectura futura propuesta*

Aspecto	Estado actual (implementado)	Evolución propuesta (no implementada)
Almacenamiento RAW	JSON en <code>data/raw/</code> , versionado en Git.	Almacén de objetos (S3, Blob, GCS) con particionamiento por fecha de ingesta y política de retención.
Base analítica	SQLite local.	Base administrada (PostgreSQL/RDS, Cloud SQL) o data warehouse.
Formato procesado	CSV.	Parquet o Delta Lake, columnar, comprimido y con esquema; particionado por <code>release_year</code> o fuente.
Procesamiento	PySpark <code>local[*]</code> .	Spark administrado (Databricks, EMR, Dataproc) con el mismo código de DataFrames.
Orquestación	GitHub Actions con <code>cron</code> .	Orquestador de datos (Airflow, Data Factory) con dependencias explícitas <code>ingesta</code> → ELT → enriquecimiento y reintentos.
Calidad	Reportes por ejecución y 40 pruebas.	Reglas de calidad automatizadas como puertas de paso, alertas y métricas históricas.
Modelo de datos	Tabla <code>games</code> + conjuntos CSV derivados.	Modelo dimensional: dimensiones de juego, plataforma y proveedor; hechos de promociones con vigencia temporal.
Gobierno	Linaje documentado en el reporte y en <code>config.py</code> .	Catálogo de datos con linaje automático, propietarios y documentación de columnas.
Seguridad	APIs públicas sin credenciales.	Gestión de secretos, control de acceso por rol y cifrado en reposo cuando se integren fuentes autenticadas.

Como prioridades, se recomienda: (a) migrar la materialización a Parquet manteniendo la misma lógica de Spark, lo que preservaría los tipos y reduciría el tamaño; (b) introducir un orquestador que encadene explícitamente las tres capas en lugar de depender de horarios escalonados; (c) conservar un histórico de las respuestas de GamerPower para convertir la fotografía de promociones en una serie temporal; (d) monitorear el esquema de las APIs y versionar los contratos de datos; (e) evolucionar hacia un modelo dimensional cuando el análisis lo requiera; y (f) incorporar gestión de secretos y control de acceso en el momento en que se integre una fuente autenticada.

Conclusiones

El proyecto construyó, de forma incremental y en un entorno local, una plataforma de datos que reproduce la estructura de una arquitectura de Big Data en la nube: zona de datos crudos, almacenamiento analítico, motor de procesamiento distribuido, zonas procesada y enriquecida, y automatización. Las tres capas conforman un solo pipeline en el que cada una consume la salida de la anterior, reutiliza módulos comunes y deja evidencias con cifras calculadas, de modo que el recorrido de cualquier registro puede seguirse desde la respuesta de la API hasta el conjunto analítico final.

La arquitectura demostró ser adecuada para el escenario planteado por tres motivos. La calidad de los datos se trata como una responsabilidad explícita: se perfila antes y después de limpiar, se documenta cada regla de imputación y tipificación y se cuantifica un puntaje reproducible. El enriquecimiento aporta valor real al análisis de mercado al integrar promociones vigentes y un catálogo de contraste, sin sacrificar la integridad del conjunto base gracias a la homologación conservadora por título, la deduplicación previa y los controles de cardinalidad. Y la automatización con pruebas y artifacts convierte el pipeline en un proceso verificable y repetible en lugar de un ejercicio manual.

El modelo de datos resultante es deliberadamente simple: una tabla física `games` en SQLite y dos conjuntos derivados con clave principal `id` y clave auxiliar `title_match`, cuyo linaje está documentado columna a columna. Esa simplicidad, junto con la separación por capas, es lo que hace que la plataforma esté preparada para las siguientes fases de análisis y visualización, y para una eventual migración a servicios administrados en la nube en la que el código de procesamiento requeriría cambios acotados. Las limitaciones identificadas (ejecución local, CSV, ausencia de

orquestador y de catálogo) no invalidan la solución, sino que delimitan con precisión el alcance de la simulación y trazan la ruta de evolución.

Referencias

Apache Software Foundation. (s.f.-a). *PySpark overview*. Apache Spark.

<https://spark.apache.org/docs/latest/api/python/>

Apache Software Foundation. (s.f.-b). *SPARK-47613: Python 3.12 removed distutils* [Reporte de incidencia]. Apache JIRA. <https://issues.apache.org/jira/browse/SPARK-47613>

Apache Software Foundation. (s.f.-c). *SPARK-44120: Remove distutils dependency* [Reporte de incidencia]. Apache JIRA. <https://issues.apache.org/jira/browse/SPARK-44120>

Apache Software Foundation. (s.f.-d). *Apache Arrow documentation*.

<https://arrow.apache.org/docs/>

FreeToGame. (s.f.). *FreeToGame API documentation*. <https://www.freetogame.com/api-doc>

GamerPower. (s.f.). *GamerPower API*. <https://www.gamerpower.com/api-read>

GitHub. (s.f.). *GitHub Actions documentation*. GitHub Docs. <https://docs.github.com/actions>

MMOBomb. (s.f.). *MMOBomb API*. <https://www.mmobomb.com/api>

pandas development team. (s.f.). *pandas documentation*. <https://pandas.pydata.org/docs/>

pytest-dev team. (s.f.). *pytest documentation*. <https://docs.pytest.org/>

Python Software Foundation. (s.f.). *Python 3 documentation*. <https://docs.python.org/3/>

Reitz, K. (s.f.). *Requests: HTTP for Humans*. <https://requests.readthedocs.io/>

SQLite Consortium. (s.f.). *SQLite documentation*. <https://www.sqlite.org/docs.html>